

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

К ЗАЩИТЕ ДОПУСТИТЬ
Руководитель курсового проекта
ассистент кафедры информатики
_____А.Г. Бокун
____.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:
«CRM-СИСТЕМА С ГРАФИЧЕСКИМ ИНТЕРФЕЙСОМ»
БГУИР КП 6-05 0612 02 073 ПЗ

Выполнил студент группы 453503
МАСЕВИЧ Полина Александровна

(подпись студента)

Курсовой проект представлен
на проверку _____.____.2026

(подпись студента)

Минск 2026

СОДЕРЖАНИЕ

Введение	5
Актуальность и цель работы	5
1 Анализ CRM-системы с точки зрения системы массового обслуживания....	6
1.1 Понятие системы массового обслуживания	6
1.2 Представление crm-системы как смо.....	7
1.3 Практическое значение использования модели смо при разработке CRM-системы.....	8
2 Анализ существующих CRM-систем	11
2.1 Обзор существующих решений	11
2.2 Анализ CRM-системы Bitrix24.....	11
2.3 Анализ CRM-системы Salesforce	12
2.4 Анализ CRM-системы Hubspot	12
2.5 Вывод по анализу аналогов	13
3 Формирование функциональных требований к системе	14
3.1 Общая характеристика разрабатываемой системы	14
3.2 Функциональные возможности системы	14
3.3 Нефункциональные требования.....	15
3.4 Ограничения и допущения	16
3.5 Вывод по разделу.....	16
4 Анализ литературы и теоретических основ объектно-ориентированного программирования.....	18
4.1 Общие принципы объектно-ориентированного программирования....	18
4.2 Инкапсуляция.....	18
4.3 Наследование	19
4.4 Полиморфизм.....	19
4.5 Абстракция	19
4.6 Архитектура mvc и её применение	20
4.7 Использование базы данных в crm-системе	20
4.8 Использование JAVA и JAVAFX.....	21
4.9 Вывод по разделу.....	21
5 Проектирование CRM-системы	22
5.1 Общая архитектура системы	22
5.2 Моделирование предметной области	22
5.4 Проектирование базы данных	24
5.5 Проектирование пользовательского интерфейса	25
5.6 Выбор технологий реализации.....	25
5.7 Вывод по разделу.....	26
6 Тестирование программного обеспечения.....	27
6.1 Цель и задачи тестирования	27
6.2 Описание процесса тестирования	27
6.3 Результаты тестирования.....	27
6.4 Вывод по тестированию.....	28

Заключение.....	30
Список использованных источников.....	31
Приложение А (обязательное) Листинг программного кода	32
Приложение Б (обязательное) блок-схема общего алгоритма работы CRM- системы	36

ВВЕДЕНИЕ

Курсовой проект посвящён разработке CRM-системы с графическим пользовательским интерфейсом на языке Java. В условиях цифровизации бизнеса организациям различных масштабов требуется использование программных средств, обеспечивающих автоматизацию процессов работы с клиентами. Эффективное ведение клиентской базы, контроль этапов сделок и систематизация контактной информации являются важными факторами повышения конкурентоспособности предприятия.

CRM-системы (Customer Relationship Management) предназначены для централизованного хранения данных о клиентах, сделках и сотрудниках, а также для упрощения процессов взаимодействия с заказчиками. Современные решения, такие как Salesforce, Bitrix24 и HubSpot, предоставляют широкий функционал для автоматизации продаж и обслуживания. Однако для учебных целей целесообразно разработать собственную упрощённую модель системы, позволяющую продемонстрировать основные этапы проектирования программного обеспечения.

Разрабатываемая в рамках курсового проекта CRM-система рассматривается как прикладное программное средство, обеспечивающее хранение и обработку данных о клиентах и сделках, изменение их статусов и удобное отображение информации с использованием графического интерфейса. При разработке особое внимание уделяется применению принципов объектно-ориентированного программирования, построению логичной архитектуры приложения и обеспечению корректной обработки пользовательских данных.

Пояснительная записка содержит описание целей и задач проекта, анализ предметной области, обоснование выбора технологий, этапы проектирования и реализации системы, а также результаты тестирования разработанного программного продукта.

АКТУАЛЬНОСТЬ И ЦЕЛЬ РАБОТЫ

Актуальность выбранной темы обусловлена тем, что эффективность работы с клиентами напрямую влияет на прибыльность, устойчивость и конкурентоспособность организации. В современных условиях рынок характеризуется высокой степенью конкуренции, поэтому качество обслуживания и скорость обработки обращений становятся ключевыми факторами успеха. Отсутствие централизованной системы хранения данных приводит к использованию разрозненных таблиц, текстовых документов и личных записей сотрудников, что существенно усложняет контроль сделок и взаимодействий с клиентами.

Целью курсового проекта является создание прототипа CRM-системы с графическим пользовательским интерфейсом на языке Java, реализующей базовый функционал учёта клиентов, клиентов, товаров, заказов, ролей пользователей и аналитических показателей. Разрабатываемое приложение должно обеспечивать удобный ввод, хранение и отображение данных, а также корректную обработку пользовательских действий.

Для достижения поставленной цели в ходе выполнения проекта предполагается решение следующих задач:

1. Проведение анализа предметной области и определение основных сущностей системы.
2. Формирование требований к функционалу разрабатываемого приложения.
3. Проектирование структуры данных и объектной модели системы.
4. Разработка архитектуры программного продукта.
5. Создание графического пользовательского интерфейса.
6. Реализация основных сценариев работы (добавление, редактирование и удаление данных).
7. Проверка работоспособности программы и тестирование ключевых функций.

Решение указанных задач позволит создать работоспособный прототип CRM-системы и продемонстрировать практическое применение принципов объектно-ориентированного программирования при разработке прикладного программного обеспечения.

1 АНАЛИЗ CRM-СИСТЕМЫ С ТОЧКИ ЗРЕНИЯ СИСТЕМЫ МАССОВОГО ОБСЛУЖИВАНИЯ

1.1 Понятие системы массового обслуживания

Система массового обслуживания (СМО) представляет собой математическую и прикладную модель, используемую для описания процессов поступления, ожидания и обработки заявок в различных технических, информационных и организационных системах. Основная задача теории массового обслуживания заключается в исследовании закономерностей функционирования систем, в которых имеется поток запросов и ограниченные ресурсы для их обработки.

СМО широко применяется при анализе работы вычислительных систем, серверов, сетевых приложений, банковских и информационных систем, а также любых программных комплексов, где необходимо обрабатывать большое количество пользовательских запросов. Использование данной модели позволяет формализовать поведение системы, выявить её узкие места и повысить эффективность функционирования.

Классическая структура системы массового обслуживания включает несколько основных компонентов:

1) Источник заявок – элемент системы, формирующий входной поток запросов. Источник может быть как детерминированным, так и случайным. В большинстве реальных систем поток заявок носит случайный характер и описывается с помощью вероятностных распределений (например, пуассоновского потока). Источником заявок могут выступать пользователи системы, внешние сервисы или автоматические процессы.

2) Входной поток заявок – последовательность запросов, поступающих в систему. Поток характеризуется интенсивностью (λ), которая показывает среднее количество заявок, поступающих за единицу времени. Кроме того, важными характеристиками являются равномерность поступления заявок и наличие пиковых нагрузок.

3) Очередь – структура, в которой заявки ожидают обслуживания в случае, если все обслуживающие устройства заняты. Очередь может быть:

- неограниченной (заявки накапливаются без ограничений);
- ограниченной (при переполнении новые заявки отклоняются).

Также различают дисциплины обслуживания:

- FIFO (first in, first out — первый пришёл, первый обслужен);
- LIFO (last in, first out);
- приоритетное обслуживание.

4) Обслуживающее устройство (канал обслуживания) — элемент системы, выполняющий обработку заявок. В информационных системах это могут быть программные модули, серверные процессы, потоки выполнения или базы данных. Количество каналов может быть:

- один (одноканальная система);

– несколько (многоканальная система).

Выходной поток заявок – поток обработанных запросов, покидающих систему после завершения обслуживания.

Функционирование системы массового обслуживания определяется рядом параметров, среди которых основными являются:

- 1) Интенсивность поступления заявок (λ) – среднее число заявок, поступающих в систему за единицу времени.
- 2) Интенсивность обслуживания (μ) – среднее число заявок, которые система может обработать за единицу времени.
- 3) Коэффициент загрузки системы (ρ), определяемый как отношение λ к μ , который показывает степень занятости обслуживающих ресурсов.

Если коэффициент загрузки приближается к единице, это означает, что система работает на пределе своих возможностей, что может привести к увеличению длины очереди и времени ожидания заявок.

Одной из ключевых задач анализа СМО является определение следующих характеристик:

- среднее время ожидания заявки в очереди;
- среднее время пребывания заявки в системе;
- средняя длина очереди;
- вероятность отказа в обслуживании;
- загрузка обслуживающих устройств.

В зависимости от структуры и параметров системы выделяют различные типы СМО. Наиболее распространённой является модель М/М/1, которая характеризуется пуассоновским входным потоком, экспоненциальным временем обслуживания и одним каналом обслуживания. Также существуют более сложные модели, такие как М/М/п (многоканальные системы) и М/М/1/К (с ограниченной длиной очереди).

Применение теории массового обслуживания позволяет не только анализировать существующие системы, но и прогнозировать их поведение при изменении нагрузки, что особенно важно при проектировании программных продуктов и информационных систем.

Таким образом, система массового обслуживания представляет собой универсальный инструмент для моделирования процессов обработки запросов и является важной частью теоретической базы при разработке современных программных решений.

1.2 Представление CRM-системы как СМО

CRM-система (Customer Relationship Management) представляет собой программный продукт, предназначенный для управления взаимодействием с клиентами, хранения и обработки информации о сделках, контактах и действиях пользователей. Несмотря на прикладной характер, подобная система может быть рассмотрена с точки зрения теории массового обслуживания.

В контексте разрабатываемой CRM-системы элементы СМО интерпретируются следующим образом:

- источник заявок – пользователи системы (менеджеры, администраторы), которые инициируют действия в системе;
- заявки – операции, выполняемые пользователями, такие как создание клиента, оформление заказа, изменение статуса сделки, просмотр информации;
- очередь – возникает при одновременном поступлении нескольких запросов, например при работе с базой данных или обработке нескольких операций;
- обслуживающее устройство – программная система, включающая бизнес-логику приложения, модули обработки данных и взаимодействия с базой данных.

Каждое действие пользователя можно рассматривать как отдельную заявку, которая поступает в систему и проходит этапы обработки. Например, при добавлении нового клиента пользователь вводит данные, после чего система выполняет проверку корректности информации, записывает данные в базу и отображает результат. Все эти этапы можно интерпретировать как процесс обслуживания заявки.

Кроме того, при работе нескольких пользователей система может испытывать повышенную нагрузку, что приводит к увеличению времени обработки запросов. В таком случае поведение системы также соответствует модели СМО, где наблюдается рост очереди и увеличение времени ожидания.

Таким образом, CRM-система представляет собой частный случай информационной системы, функционирование которой может быть описано с использованием моделей массового обслуживания.

1.3 Практическое значение использования модели СМО при разработке CRM-системы

Рассмотрение CRM-системы с точки зрения системы массового обслуживания имеет важное теоретическое и практическое значение при проектировании, разработке и анализе программного продукта. Несмотря на то, что CRM-системы в первую очередь являются прикладными информационными системами, применение моделей СМО позволяет более глубоко понять процессы их функционирования и повысить эффективность реализации.

Прежде всего, использование модели СМО позволяет формализовать процессы взаимодействия пользователя с системой. Любое действие пользователя – добавление клиента, создание заказа, редактирование информации или выполнение поиска – может быть интерпретировано как заявка, поступающая в систему на обработку. Такой подход позволяет представить работу CRM-системы в виде последовательности операций, каждая из которых проходит этапы поступления, ожидания и обработки. Это

существенно упрощает анализ поведения системы и делает её функционирование более прозрачным с инженерной точки зрения.

Кроме того, применение концепции СМО позволяет учитывать влияние нагрузки на систему. В реальных условиях CRM-система может использоваться одновременно несколькими пользователями, что приводит к увеличению количества поступающих запросов. При этом ресурсы системы (процессор, оперативная память, каналы ввода-вывода, база данных) являются ограниченными, что может вызывать задержки в обработке данных. Рассмотрение системы как СМО позволяет прогнозировать подобные ситуации, анализировать поведение системы при увеличении интенсивности входного потока заявок и принимать решения, направленные на повышение производительности.

Важным аспектом является возможность выявления узких мест системы. Например, при работе с базой данных могут возникать задержки, связанные с выполнением сложных запросов или одновременным доступом нескольких пользователей к одним и тем же данным. С точки зрения СМО это соответствует ситуации перегрузки обслуживающего устройства, при которой увеличивается длина очереди и время ожидания. Анализ подобных ситуаций позволяет оптимизировать структуру базы данных, улучшить алгоритмы обработки информации и повысить общую эффективность системы.

Использование модели массового обслуживания также способствует обоснованному выбору архитектурных решений. При проектировании CRM-системы разработчик может учитывать необходимость разделения системы на логические уровни, такие как пользовательский интерфейс, бизнес-логика и уровень доступа к данным. Такое разделение (например, в рамках архитектуры MVC) позволяет распределить нагрузку между компонентами системы и снизить вероятность возникновения «узких мест». С точки зрения СМО это можно рассматривать как организацию нескольких обслуживающих каналов, работающих параллельно.

Ещё одним важным преимуществом применения модели СМО является возможность повышения масштабируемости системы. При увеличении количества пользователей или объёма данных система должна сохранять приемлемую производительность. Анализ с использованием СМО позволяет заранее предусмотреть потенциальные проблемы, связанные с ростом нагрузки, и реализовать решения, направленные на их устранение. К таким решениям можно отнести оптимизацию запросов к базе данных, использование кэширования, а также распределение обработки данных между несколькими потоками выполнения.

Следует отметить, что в рамках данного курсового проекта не предполагается проведение строгого математического анализа параметров системы массового обслуживания, такого как вычисление вероятностей состояний системы или аналитическое определение времени ожидания. Однако даже на концептуальном уровне использование данной модели

позволяет значительно повысить качество проектирования и обоснованность принимаемых решений.

Дополнительно, применение подходов СМО способствует более глубокому пониманию процессов взаимодействия пользователя с программным продуктом. Это особенно важно при разработке интерфейса, поскольку позволяет учитывать возможные задержки в отклике системы и проектировать интерфейс таким образом, чтобы обеспечить максимальное удобство и комфорт работы пользователя. Например, можно предусмотреть визуальные индикаторы загрузки, блокировку интерфейса на время выполнения операций или асинхронную обработку запросов.

Также стоит отметить, что использование концепции СМО позволяет рассматривать CRM-систему не только как набор отдельных функций, но и как целостную систему обработки данных, в которой важны не только отдельные операции, но и их взаимодействие, последовательность выполнения и влияние друг на друга. Это способствует формированию системного мышления при разработке программного обеспечения, что является одной из ключевых целей изучения объектно-ориентированного программирования.

Таким образом, применение модели системы массового обслуживания при разработке CRM-системы позволяет:

- формализовать процессы обработки пользовательских запросов;
- учитывать влияние нагрузки на систему;
- выявлять и устранять узкие места;
- обосновывать архитектурные решения;
- повышать производительность и масштабируемость;
- улучшать взаимодействие пользователя с системой.

В результате можно сделать вывод о том, что использование принципов теории массового обслуживания является полезным инструментом при проектировании CRM-системы, позволяющим повысить качество разрабатываемого программного продукта и обеспечить более эффективную организацию процессов обработки данных.

2 АНАЛИЗ СУЩЕСТВУЮЩИХ CRM-СИСТЕМ

2.1 Обзор существующих решений

В условиях активного развития информационных технологий и цифровизации бизнеса особую значимость приобретают программные системы, направленные на автоматизацию взаимодействия с клиентами. К числу таких систем относятся CRM-системы (Customer Relationship Management), которые используются для хранения, обработки и анализа данных о клиентах, а также для управления процессами продаж и обслуживания.

Современный рынок программного обеспечения предлагает широкий спектр CRM-решений, отличающихся по функциональности, сложности и области применения. Существующие системы могут быть ориентированы как на малый бизнес, так и на крупные корпорации, обеспечивая различные уровни автоматизации и интеграции с другими сервисами.

Большинство CRM-систем предоставляют пользователям возможность централизованного хранения информации о клиентах, ведения истории взаимодействий, управления сделками, а также анализа эффективности работы сотрудников. При этом каждая система имеет свои особенности, преимущества и ограничения, которые необходимо учитывать при разработке собственного программного продукта.

Для более детального анализа были рассмотрены наиболее популярные CRM-системы, получившие широкое распространение на практике: Bitrix24, Salesforce и HubSpot.

2.2 Анализ CRM-системы Bitrix24

Bitrix24 является одной из наиболее известных CRM-систем, широко применяемых в странах СНГ. Данная система представляет собой комплексное решение для управления бизнес-процессами, объединяющее в себе инструменты для работы с клиентами, задачами, проектами и внутренними коммуникациями.

Одной из ключевых особенностей Bitrix24 является его многофункциональность. Пользователям предоставляется возможность не только вести клиентскую базу и отслеживать сделки, но и организовывать совместную работу сотрудников, использовать встроенные средства коммуникации, а также автоматизировать различные бизнес-процессы. Это делает систему универсальным инструментом для управления деятельностью организации.

Однако высокая функциональность системы имеет и обратную сторону. Интерфейс Bitrix24 может показаться перегруженным, особенно для новых пользователей, не имеющих опыта работы с подобными системами. Кроме

того, для эффективного использования всех возможностей системы требуется определённое время на обучение и настройку.

Следует также отметить, что, несмотря на наличие бесплатной версии, многие функции доступны только в платных тарифах, что может ограничивать использование системы в небольших компаниях или в рамках учебных проектов.

Таким образом, Vitrix24 является мощным инструментом, однако его избыточная сложность и перегруженность функционалом делают его не самым удобным решением для демонстрации базовых принципов работы CRM-систем.

2.3 Анализ CRM-системы Salesforce

Salesforce занимает лидирующие позиции на мировом рынке CRM-систем и широко используется крупными компаниями для управления взаимоотношениями с клиентами. Данная платформа отличается высокой гибкостью и масштабируемостью, что позволяет адаптировать её под различные бизнес-задачи.

Одной из ключевых особенностей Salesforce является возможность глубокой настройки системы под конкретные потребности организации. Пользователи могут изменять структуру данных, создавать собственные бизнес-процессы и интегрировать систему с другими корпоративными решениями. Кроме того, Salesforce предоставляет развитые инструменты аналитики, позволяющие оценивать эффективность продаж и прогнозировать результаты деятельности.

Несмотря на очевидные преимущества, система обладает рядом существенных недостатков. Прежде всего, это высокая стоимость использования, которая делает её недоступной для малого бизнеса и учебных проектов. Кроме того, внедрение и настройка Salesforce требуют значительных временных и трудовых затрат, а также наличия квалифицированных специалистов.

Сложность интерфейса и большое количество функций также могут затруднять освоение системы пользователями без соответствующего опыта. В связи с этим использование Salesforce в рамках курсового проекта является нецелесообразным, однако анализ её возможностей позволяет выделить ключевые функции, которые могут быть реализованы в упрощённом виде.

2.4 Анализ CRM-системы HubSpot

HubSpot представляет собой современную CRM-систему, ориентированную на автоматизацию маркетинга, продаж и обслуживания клиентов. В отличие от более сложных решений, данная система делает акцент на удобстве использования и доступности для широкого круга пользователей.

Интерфейс HubSpot отличается простотой и интуитивной понятностью,

что позволяет быстро освоить основные функции системы без необходимости длительного обучения. Пользователям предоставляется возможность вести базу клиентов, отслеживать сделки, а также использовать инструменты для анализа эффективности маркетинговых кампаний.

Одним из преимуществ системы является наличие бесплатной версии, которая включает базовый функционал и может использоваться небольшими компаниями. Однако расширенные возможности доступны только в платных тарифах, стоимость которых может быть достаточно высокой.

Следует отметить, что HubSpot в большей степени ориентирован на маркетинговые задачи, что может ограничивать его использование в качестве универсальной CRM-системы. Кроме того, зависимость от облачной инфраструктуры требует постоянного подключения к сети Интернет.

Таким образом, HubSpot является удобным и доступным решением, однако его функциональность ограничена по сравнению с более сложными системами.

2.5 Вывод по анализу аналогов

Проведённый анализ показал, что современные CRM-системы обладают широкими возможностями и позволяют эффективно автоматизировать процессы взаимодействия с клиентами. Каждая из рассмотренных систем имеет свои особенности, которые делают её подходящей для определённых задач и условий использования.

В то же время можно выделить ряд общих недостатков, характерных для большинства существующих решений. К ним относятся высокая сложность систем, необходимость длительного обучения пользователей, избыточность функционала для простых задач, а также высокая стоимость использования.

Для целей учебного проекта разработка собственной CRM-системы является обоснованной, поскольку позволяет сосредоточиться на реализации базового функционала без излишнего усложнения. При этом создаваемое приложение может учитывать положительный опыт существующих систем, таких как централизованное хранение данных, управление сделками и удобный пользовательский интерфейс.

Таким образом, анализ аналогов позволил определить основные требования к разрабатываемой системе и сформировать представление о её функциональности и структуре. Разрабатываемая CRM-система будет представлять собой упрощённую модель, ориентированную на демонстрацию принципов объектно-ориентированного программирования и удобство использования.

3 ФОРМИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ К СИСТЕМЕ

3.1 Общая характеристика разрабатываемой системы

На основании проведённого анализа существующих CRM-систем и особенностей предметной области можно сформулировать основные требования к разрабатываемому программному продукту. Разрабатываемая система представляет собой приложение с графическим пользовательским интерфейсом, предназначенное для управления клиентской базой и обработки информации о заказах.

Система ориентирована на использование в условиях малого или учебного бизнеса и должна обеспечивать удобное взаимодействие пользователя с данными. Основной задачей является реализация базового функционала CRM-системы без избыточной сложности, характерной для коммерческих решений.

Разрабатываемое приложение должно обеспечивать хранение, обработку и отображение информации, а также поддерживать основные операции по управлению данными. При этом особое внимание должно уделяться простоте интерфейса, логичности структуры и корректной обработке пользовательских действий.

3.2 Функциональные возможности системы

Таблица 3.1 – Реализованные функциональные возможности CRM-системы

Подсистема	Реализованные возможности
Авторизация	вход пользователей с ролями ADMIN и MANAGER
Клиенты	добавление, изменение, удаление, просмотр ФИО, телефона, адреса и суммы выкупа
Каталог товаров	добавление, редактирование цены и удаление товаров
Заказы	выбор клиента и товара, статус, доставка, адрес, комментарий, итоговая сумма
Скидки	автоматический расчёт скидки по сумме выкупа клиента
Аналитика	общая выручка, количество заказов, средний чек, популярный товар

Функциональные требования определяют перечень возможностей, которые должна предоставлять система пользователю. В рамках данного курсового проекта предполагается реализация базового набора функций, обеспечивающих полноценную работу с клиентами и заказами.

Одной из ключевых функций системы является управление пользовательскими данными. Система должна обеспечивать возможность регистрации и авторизации пользователей, а также разграничение прав доступа. В зависимости от роли пользователя (например, администратор или менеджер) может предоставляться различный набор возможностей.

Важным элементом является работа с клиентской базой. Пользователь должен иметь возможность добавлять новых клиентов, редактировать существующую информацию, а также удалять записи при необходимости. Кроме того, система должна обеспечивать удобное отображение списка клиентов и возможность поиска по различным параметрам, таким как имя, контактные данные или другие характеристики.

Не менее важной функцией является управление заказами. Система должна предоставлять возможность создания заказов, привязанных к конкретным клиентам, а также изменения их статуса. Например, заказ может находиться в состоянии «новый», «в обработке» или «завершён». Пользователь должен иметь возможность отслеживать текущее состояние заказов и при необходимости вносить изменения.

Также система должна обеспечивать отображение информации в удобном и наглядном виде. Для этого могут использоваться таблицы, формы и другие элементы графического интерфейса. Пользователь должен иметь возможность быстро получать доступ к необходимой информации и выполнять основные операции без лишних действий.

Дополнительно может быть реализована функция фильтрации и сортировки данных, что позволит упростить работу с большим количеством записей. Это особенно важно при увеличении объёма информации в системе.

3.3 Нефункциональные требования

Помимо функциональных возможностей, система должна соответствовать ряду нефункциональных требований, определяющих качество её работы и удобство использования.

Одним из ключевых требований является удобство пользовательского интерфейса. Интерфейс должен быть интуитивно понятным и не требовать длительного обучения. Все основные функции должны быть доступны в несколько действий, а структура приложения должна быть логичной и последовательной.

Важным аспектом является производительность системы. Приложение должно обеспечивать быструю обработку пользовательских запросов и минимальное время отклика при выполнении операций. Это особенно актуально при работе с базой данных и обработке большого количества

записей.

Также необходимо учитывать надёжность системы. Приложение должно корректно обрабатывать ошибки, возникающие при вводе данных или работе с базой данных, и предотвращать потерю информации. При возникновении ошибок пользователю должны выводиться понятные сообщения.

Дополнительно следует отметить требование к масштабируемости. Несмотря на то, что система разрабатывается в учебных целях, её архитектура должна предусматривать возможность расширения функциональности в будущем. Это может включать добавление новых модулей, расширение структуры данных или интеграцию с другими системами.

3.4 Ограничения и допущения

При разработке CRM-системы необходимо учитывать ряд ограничений, обусловленных учебным характером проекта.

Во-первых, система разрабатывается как локальное приложение, не требующее подключения к удалённым серверам. Это упрощает реализацию и позволяет сосредоточиться на изучении принципов объектно-ориентированного программирования.

Во-вторых, реализуется только базовый функционал, необходимый для демонстрации работы системы. Расширенные возможности, такие как интеграция с внешними сервисами или сложная аналитика, в рамках проекта не рассматриваются.

В-третьих, предполагается использование встроенной или локальной базы данных, что также упрощает процесс разработки и тестирования.

Несмотря на указанные ограничения, система должна быть реализована с соблюдением принципов ООП и иметь чёткую архитектуру, что позволит продемонстрировать основные навыки проектирования программного обеспечения.

3.5 Вывод по разделу

В результате проведённого анализа и формирования функциональных требований были определены ключевые возможности и характеристики разрабатываемой CRM-системы. Установлено, что система должна обеспечивать эффективное управление клиентской базой и заказами, поддерживать основные операции по добавлению, редактированию и удалению данных, а также предоставлять пользователю удобный и интуитивно понятный графический интерфейс.

Особое внимание в рамках данного раздела было уделено не только определению функциональных возможностей системы, но и формированию нефункциональных требований, таких как удобство использования, производительность, надёжность и масштабируемость. Учет данных

требований позволяет обеспечить более качественную реализацию программного продукта и повысить его практическую ценность.

Сформулированные требования являются основой для последующих этапов разработки, включая проектирование архитектуры системы, выбор технологий и реализацию программных компонентов. Они определяют структуру будущего приложения, его основные модули и принципы взаимодействия между ними.

Кроме того, чёткое определение требований позволяет снизить вероятность ошибок на этапе реализации, а также обеспечивает более системный подход к разработке программного обеспечения. Это особенно важно при использовании принципов объектно-ориентированного программирования, где правильное проектирование структуры классов и их взаимодействия играет ключевую роль.

Таким образом, данный раздел закладывает фундамент для дальнейшей разработки CRM-системы и обеспечивает логическую связь между теоретической частью работы и практической реализацией программного продукта.

4 АНАЛИЗ ЛИТЕРАТУРЫ И ТЕОРЕТИЧЕСКИХ ОСНОВ ОБЪЕКТНО-ОРИЕНТИРОВАННОГО ПРОГРАММИРОВАНИЯ

4.1 Общие принципы объектно-ориентированного программирования

Объектно-ориентированное программирование (ООП) является одной из наиболее распространённых парадигм разработки программного обеспечения, широко применяемой при создании современных информационных систем. Основной идеей ООП является представление программы в виде совокупности взаимодействующих объектов, каждый из которых обладает собственным состоянием и поведением.

В отличие от процедурного подхода, где основное внимание уделяется последовательности выполняемых операций, объектно-ориентированное программирование фокусируется на моделировании реальных сущностей предметной области. Это позволяет создавать более гибкие, расширяемые и удобные для сопровождения программные системы.

Ключевыми понятиями ООП являются класс и объект. Класс представляет собой абстрактное описание сущности, определяющее её свойства (поля) и поведение (методы). Объект, в свою очередь, является конкретным экземпляром класса, обладающим определённым состоянием.

Использование классов и объектов позволяет структурировать программный код, разделяя его на логические компоненты. Это упрощает разработку, тестирование и дальнейшую модификацию программного продукта.

Объектно-ориентированный подход особенно эффективен при разработке сложных систем, таких как CRM-системы, где необходимо учитывать большое количество взаимосвязанных сущностей, например пользователей, клиентов, заказов и других объектов.

4.2 Инкапсуляция

Инкапсуляция является одним из базовых принципов объектно-ориентированного программирования и заключается в объединении данных и методов их обработки в рамках одного класса, а также в ограничении доступа к внутреннему состоянию объекта.

Основной целью инкапсуляции является защита данных от некорректного использования и обеспечение контроля над их изменением. Это достигается за счёт использования модификаторов доступа, таких как `private`, `protected` и `public`.

В рамках CRM-системы принцип инкапсуляции может применяться, например, при работе с данными клиента. Поля, содержащие информацию о клиенте, такие как имя, контактные данные или история заказов, могут быть объявлены как закрытые (`private`), а доступ к ним осуществляется через

специальные методы – геттеры и сеттеры.

Такой подход позволяет контролировать изменения данных и предотвращать их некорректное использование. Например, при установке значения поля можно добавить проверку корректности введенной информации.

Инкапсуляция способствует повышению надёжности программного продукта и упрощает его сопровождение, поскольку изменения внутренней реализации класса не влияют на внешний код, использующий данный класс.

4.3 Наследование

Наследование является механизмом, позволяющим создавать новые классы на основе уже существующих. При этом новый класс (подкласс) наследует свойства и методы базового класса, а также может расширять или изменять их.

Данный принцип позволяет повторно использовать код и уменьшать его дублирование. Наследование также способствует созданию иерархий классов, отражающих структуру предметной области.

В контексте CRM-системы наследование может применяться, например, для описания различных типов пользователей. Можно выделить базовый класс *User*, содержащий общие свойства, такие как логин и пароль, и создать на его основе подклассы *Admin* и *Manager*, которые будут обладать дополнительными возможностями.

Использование наследования позволяет упростить структуру программы и сделать её более логичной и понятной.

4.4 Полиморфизм

Полиморфизм представляет собой возможность использования объектов разных классов через единый интерфейс. Это означает, что один и тот же метод может вести себя по-разному в зависимости от объекта, к которому он применяется.

Полиморфизм позволяет создавать более гибкие и расширяемые системы, поскольку новые классы могут быть добавлены без изменения существующего кода.

В CRM-системе полиморфизм может использоваться, например, при обработке различных типов пользователей или заказов. Один и тот же метод может выполнять разные действия в зависимости от типа объекта.

Это позволяет упростить код и повысить его читаемость, а также облегчает добавление новых функций в будущем.

4.5 Абстракция

Абстракция заключается в выделении наиболее значимых

характеристик объекта и игнорировании несущественных деталей. В объектно-ориентированном программировании это достигается с помощью абстрактных классов и интерфейсов.

Абстракция позволяет создавать обобщённые модели, которые могут быть использованы для описания различных объектов. Это упрощает проектирование системы и делает её более гибкой.

В CRM-системе абстракция может применяться, например, при описании сущности пользователя. Можно определить абстрактный класс User, который будет содержать общие характеристики, а конкретные типы пользователей будут реализовывать специфические особенности.

Использование абстракции позволяет уменьшить сложность системы и повысить её масштабируемость.

4.6 Архитектура MVC и её применение

Одним из наиболее распространённых подходов к организации структуры программного обеспечения является архитектура MVC (Model–View–Controller). Данный подход предполагает разделение приложения на три основных компонента: модель, представление и контроллер.

Модель отвечает за хранение данных и бизнес-логику приложения. Представление обеспечивает отображение информации пользователю, а контроллер обрабатывает действия пользователя и управляет взаимодействием между моделью и представлением.

В разрабатываемой CRM-системе архитектура MVC используется для разделения логики приложения и пользовательского интерфейса. Это позволяет сделать код более структурированным и удобным для сопровождения. Например, изменения в интерфейсе не требуют изменения логики обработки данных, и наоборот.

Применение архитектуры MVC также способствует повышению читаемости кода и облегчает его тестирование.

4.7 Использование базы данных в CRM-системе

Хранение данных является важной частью любой CRM-системы. Для этого используются базы данных, позволяющие организовать надёжное и структурированное хранение информации.

В рамках данного проекта предполагается использование локальной базы данных, которая обеспечивает хранение информации о клиентах, заказах и пользователях. Взаимодействие с базой данных осуществляется с помощью соответствующих средств языка программирования.

Использование базы данных позволяет обеспечить сохранность информации, а также реализовать операции поиска, сортировки и фильтрации данных. Это значительно повышает удобство работы с системой и её функциональность.

4.8 Использование Java и JavaFX

Для реализации CRM-системы выбран язык программирования Java, который является одним из наиболее популярных и широко используемых языков. Он обеспечивает надёжность, переносимость и развитую экосистему инструментов.

Для создания графического пользовательского интерфейса используется технология JavaFX, которая позволяет разрабатывать современные приложения с удобным и функциональным интерфейсом. JavaFX предоставляет широкий набор компонентов для отображения данных, обработки событий и создания интерактивных элементов.

Использование Java и JavaFX позволяет реализовать полноценное приложение с графическим интерфейсом, соответствующее требованиям курсового проекта.

4.9 Вывод по разделу

В данном разделе были рассмотрены основные теоретические положения объектно-ориентированного программирования, а также их применение в контексте разработки CRM-системы. Были проанализированы ключевые принципы ООП, такие как инкапсуляция, наследование, полиморфизм и абстракция, которые лежат в основе построения современных программных систем.

Особое внимание было уделено практическому применению данных принципов при разработке программного продукта. Показано, что использование объектно-ориентированного подхода позволяет эффективно моделировать предметную область, обеспечивать гибкость системы и упрощать её сопровождение.

Также была рассмотрена архитектура MVC, позволяющая структурировать приложение и разделить его на логические компоненты, что способствует повышению качества программного кода. Дополнительно были затронуты вопросы хранения данных и выбора технологий, используемых при реализации системы.

Таким образом, проведённый анализ теоретических основ позволяет сделать вывод о том, что применение принципов объектно-ориентированного программирования является обоснованным и необходимым при разработке CRM-системы. Полученные теоретические знания будут использованы на следующем этапе – проектировании архитектуры и реализации программного продукта.

5 ПРОЕКТИРОВАНИЕ CRM-СИСТЕМЫ

5.1 Общая архитектура системы

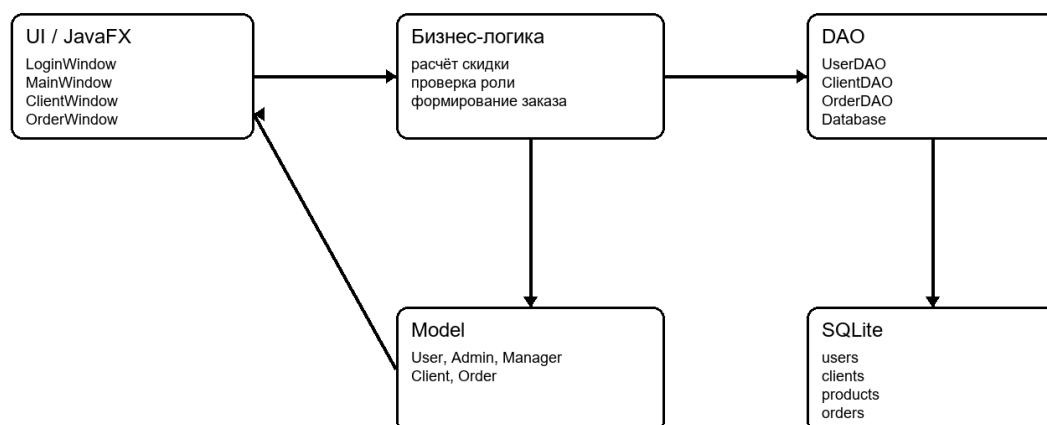


Рисунок 5.1 – Общая архитектура CRM-системы

На этапе проектирования определяется структура разрабатываемого программного продукта, а также способы взаимодействия его компонентов. В рамках данного курсового проекта для построения системы выбрана архитектура MVC (Model–View–Controller), которая позволяет разделить приложение на логические части и упростить его разработку.

Архитектура MVC предполагает разделение системы на три основных компонента. Модель отвечает за хранение данных и реализацию бизнес-логики. Представление обеспечивает отображение информации пользователю, а контроллер обрабатывает действия пользователя и управляет взаимодействием между моделью и интерфейсом.

В разрабатываемой CRM-системе данный подход позволяет отделить пользовательский интерфейс от логики обработки данных. Это делает код более структурированным и удобным для сопровождения. Например, изменение интерфейса не требует изменения логики работы с данными, что повышает гибкость системы.

Кроме того, использование архитектуры MVC способствует более чёткому распределению ответственности между компонентами системы, что является важным принципом объектно-ориентированного проектирования.

5.2 Моделирование предметной области

При разработке CRM-системы необходимо определить основные сущности, которые будут использоваться в программе. Эти сущности отражают реальные объекты предметной области и реализуются в виде

классов.

Ключевыми элементами системы являются пользователь, клиент, товар и заказ. Пользователь представляет собой лицо, работающее с системой, и может иметь различные роли. Клиент содержит информацию о заказчиках, с которыми взаимодействует организация. Заказ отражает конкретную сделку или операцию, связанную с клиентом.

Каждая из указанных сущностей обладает набором свойств и методов. Например, клиент может содержать имя, контактные данные и список заказов, а заказ — описание, дату создания и статус выполнения.

Связи между сущностями также играют важную роль. Один клиент может иметь несколько заказов, что реализуется с помощью коллекций. Пользователь, в свою очередь, выполняет операции над клиентами и заказами.

Такое моделирование позволяет отразить структуру предметной области и создать основу для реализации системы.

5.3 Проектирование классов системы

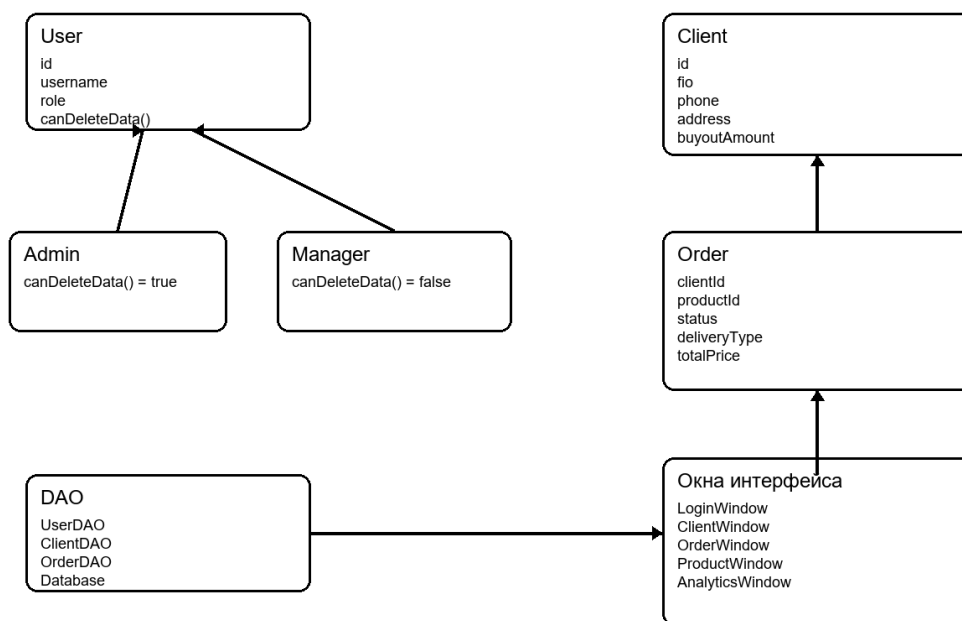


Рисунок 5.2 – Основные классы и связи CRM-системы

На основе анализа предметной области формируется структура классов, составляющих основу программного продукта.

В системе выделяется базовый класс пользователя, содержащий общие характеристики, такие как логин и пароль. На его основе могут быть реализованы специализированные классы, отражающие различные роли пользователей.

Класс клиента предназначен для хранения информации о заказчиках. Он включает поля, содержащие основные данные, а также методы для работы с этой информацией.

Класс заказа описывает отдельную операцию покупки товара клиентом.

В нём содержатся данные о клиенте, статусе выполнения и других характеристиках.

Дополнительно может быть реализован класс, отвечающий за управление данными, например взаимодействие с базой данных. Это позволяет отделить логику хранения информации от основной функциональности системы.

При проектировании классов особое внимание уделяется принципам объектно-ориентированного программирования. Используется инкапсуляция для защиты данных, наследование для упрощения структуры и полиморфизм для обеспечения гибкости системы.

5.4 Проектирование базы данных

Таблица 5.1 – Структура базы данных CRM-системы

Таблица	Назначение	Основные поля
users	хранение учётных записей	id, username, password, role
clients	хранение данных клиентов	id, fio, phone, address, buyout_amount
products	каталог товаров	id, name, price
orders	хранение заказов	id, client_id, product_id, status, delivery_type, total_price, comment

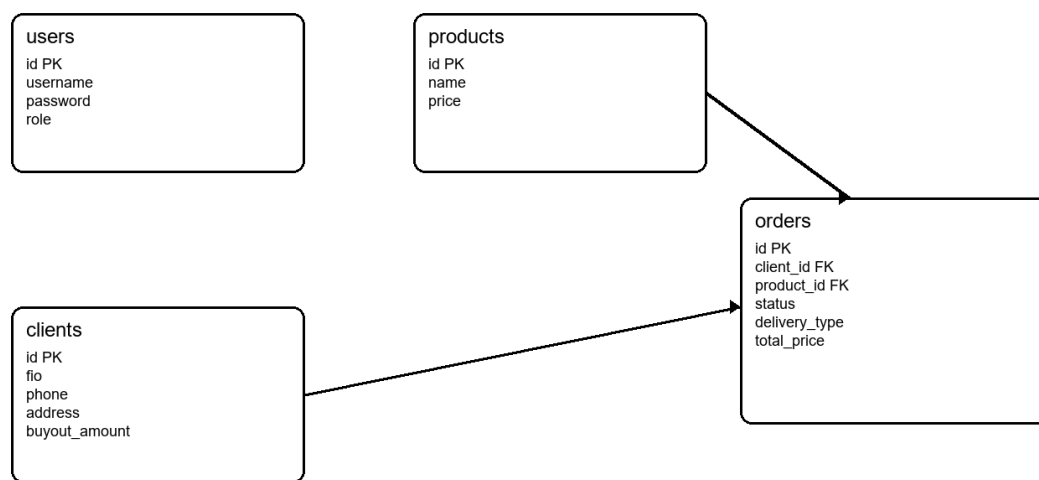


Рисунок 5.3 – Логическая схема базы данных

Для хранения информации в CRM-системе используется база данных. На этапе проектирования определяется структура таблиц и их взаимосвязи.

Основными таблицами являются таблицы пользователей, клиентов, товаров и заказов. Таблица пользователей содержит данные для авторизации

и определения ролей. Таблица клиентов хранит информацию о заказчиках, а таблица заказов – данные о сделках.

Между таблицами устанавливаются связи. Например, таблица заказов связана с таблицей клиентов, что позволяет определить, к какому клиенту относится конкретный заказ.

Структура базы данных должна обеспечивать удобное хранение информации и эффективный доступ к ней. Это позволяет реализовать основные операции, такие как добавление, изменение и удаление данных.

5.5 Проектирование пользовательского интерфейса

Таблица 5.2 – Назначение окон пользовательского интерфейса

Окно	Назначение
LoginWindow	авторизация пользователя и запуск сессии
MainWindow	главная панель навигации по модулям
ClientWindow	управление клиентской базой
ProductWindow	управление каталогом товаров
OrderWindow	создание заказов и просмотр истории
AnalyticsWindow	просмотр базовых аналитических показателей

Пользовательский интерфейс играет важную роль в системе, поскольку обеспечивает взаимодействие пользователя с программой. В рамках данного проекта интерфейс реализуется с использованием технологии JavaFX.

Интерфейс включает основные элементы, такие как формы ввода данных, таблицы для отображения информации и элементы управления. Пользователь должен иметь возможность выполнять основные операции, не затрачивая значительных усилий.

При проектировании интерфейса учитываются принципы удобства использования. Все элементы должны быть расположены логично, а действия пользователя – понятны и предсказуемы. Это позволяет повысить эффективность работы с системой.

5.6 Выбор технологий реализации

Листинг основных программных модулей CRM-системы приведён в приложении А. Блок-схема общего алгоритма работы приложения приведена в приложении Б.

Для разработки CRM-системы выбран язык программирования Java, который обеспечивает надёжность и широкие возможности для создания приложений. Использование Java позволяет реализовать объектно-

ориентированный подход и эффективно работать с данными.

Для создания графического интерфейса используется JavaFX, предоставляющий необходимые инструменты для разработки современных приложений. Для хранения данных применяется локальная база данных, что упрощает процесс разработки и тестирования.

Выбор указанных технологий обусловлен их доступностью, функциональностью и соответствием требованиям курсового проекта.

5.7 Вывод по разделу

В данном разделе было выполнено проектирование CRM-системы, включающее определение архитектуры, моделирование предметной области и разработку структуры классов. Было показано, что использование архитектуры MVC позволяет эффективно разделить систему на логические компоненты и упростить её разработку.

Также была определена структура базы данных и основные принципы организации пользовательского интерфейса. Выбор технологий реализации был обоснован с точки зрения их функциональности и соответствия поставленным задачам.

Таким образом, этап проектирования позволяет создать основу для реализации программного продукта и обеспечивает логическую связь между теоретической частью работы и практической реализацией системы.

6 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

6.1 Цель и задачи тестирования

Тестирование программного обеспечения является важным этапом разработки, направленным на проверку корректности работы системы и выявление возможных ошибок. В рамках данного курсового проекта тестирование проводится с целью проверки работоспособности разработанной CRM-системы и подтверждения соответствия её функциональности ранее сформулированным требованиям.

Основной задачей тестирования является проверка корректности выполнения основных операций, таких как работа с клиентами, управление заказами и взаимодействие пользователя с интерфейсом системы.

6.2 Описание процесса тестирования

Тестирование разработанной CRM-системы проводилось путём выполнения различных пользовательских сценариев, отражающих основные способы взаимодействия с программой. Проверка выполнялась в условиях, приближённых к реальной эксплуатации системы.

В ходе тестирования особое внимание уделялось корректности обработки пользовательских действий, а также устойчивости системы к возможным ошибкам ввода данных. Также проверялась работа интерфейса и корректность отображения информации.

6.3 Результаты тестирования

Таблица 6.1 – Результаты тестирования CRM-системы

Сценарий	Ожидаемый результат	Фактический результат
Авторизация администратора	открывается главное окно с ролью ADMIN	выполнено
Авторизация менеджера	открывается главное окно с ролью MANAGER	выполнено
Добавление клиента	новая запись появляется в таблице клиентов	выполнено
Редактирование клиента	изменённые данные сохраняются в SQLite	выполнено
Добавление товара	товар отображается в каталоге и доступен при заказе	выполнено

Продолжение таблицы 6.1

Сценарий	Ожидаемый результат	Фактический результат
Создание заказа	заказ сохраняется с клиентом, товаром, статусом и суммой	выполнено
Расчёт скидки	итоговая цена уменьшается согласно сумме выкупа	выполнено
Окно аналитики	отображаются выручка, количество заказов и средний чек	выполнено

В процессе тестирования были проверены основные функции системы. Пользователь имел возможность успешно выполнять операции по добавлению, редактированию и удалению клиентов. Все изменения корректно сохранялись и отображались в интерфейсе.

Также была проверена работа с заказами. Система корректно обрабатывала создание новых заказов, изменение их статуса и отображение связанной информации. Ошибки при выполнении данных операций не выявлены.

Дополнительно была проведена проверка корректности работы интерфейса. Все элементы управления функционировали правильно, а отображаемая информация соответствовала введённым данным.

В ходе тестирования не было выявлено критических ошибок, влияющих на работоспособность системы. Незначительные недочёты, связанные с отображением или вводом данных, могут быть устранены в процессе дальнейшей доработки.

6.4 Вывод по тестированию

В результате проведённого тестирования было установлено, что разработанная CRM-система корректно выполняет все основные функции, предусмотренные на этапе формирования требований. Были успешно проверены ключевые сценарии работы пользователя, включая добавление, редактирование и удаление клиентов, создание и обработку заказов, а также взаимодействие с пользовательским интерфейсом.

Особое внимание в процессе тестирования уделялось устойчивости системы к различным типам пользовательского ввода. В ходе проверки было подтверждено, что система корректно обрабатывает вводимые данные, предотвращает выполнение некорректных операций и обеспечивает сохранность информации. Это свидетельствует о достаточном уровне надёжности разработанного программного продукта.

Также была проведена оценка удобства использования системы. Интерфейс приложения показал себя как логичный и интуитивно понятный, что позволяет пользователю эффективно выполнять основные операции без

необходимости длительного обучения. Все элементы управления функционируют корректно, а отображаемая информация соответствует текущему состоянию данных.

Дополнительно была проверена производительность системы при выполнении типовых операций. Время отклика приложения находится на приемлемом уровне, что обеспечивает комфортную работу пользователя. Даже при увеличении количества записей система сохраняет стабильность и не демонстрирует значительных задержек.

В ходе тестирования не было выявлено критических ошибок, способных привести к сбоям в работе системы или потере данных. Обнаруженные незначительные недочёты носят локальный характер и могут быть устранены в процессе дальнейшей доработки приложения.

Таким образом, результаты тестирования подтверждают, что разработанная CRM-система является работоспособной, стабильной и соответствует поставленным требованиям. Программный продукт может быть использован для решения задач управления клиентской базой и заказами, а также служит наглядным примером применения принципов объектно-ориентированного программирования при разработке прикладных систем.

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта была разработана CRM-система, предназначенная для управления клиентской базой, каталогом товаров, заказами и аналитическими показателями. Основной целью работы являлось применение принципов объектно-ориентированного программирования при создании прикладного программного продукта с графическим пользовательским интерфейсом.

В процессе выполнения работы были изучены теоретические основы систем массового обслуживания, что позволило рассмотреть CRM-систему как систему обработки пользовательских запросов и лучше понять процессы её функционирования.

Также был проведён анализ существующих CRM-систем, в результате которого были выявлены их основные преимущества и недостатки. Это позволило сформировать требования к разрабатываемой системе и определить её функциональные возможности.

На этапе проектирования была выбрана архитектура MVC, разработана структура классов и спроектирована база данных. В ходе реализации были использованы язык программирования Java и технология JavaFX, что позволило создать приложение с графическим пользовательским интерфейсом.

Проведённое тестирование подтвердило корректность работы системы и соответствие её функциональности поставленным требованиям. Были успешно проверены основные сценарии использования, а также оценена стабильность работы приложения.

Практическая значимость работы заключается в том, что разработанная CRM-система может быть использована в учебных целях и демонстрирует применение объектно-ориентированного подхода при разработке программного обеспечения.

Таким образом, поставленные цели и задачи курсового проекта были успешно достигнуты.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Гради Буч, Джеймс Рамбо, Айвар Якобсон. Объектно-ориентированный анализ и проектирование. – М.: Вильямс, 2018.
- [2] Роберт Мартин. Чистая архитектура: искусство разработки программного обеспечения. – СПб.: Питер, 2020.
- [3] Эрих Гамма, Ричард Хелм, Ральф Джонсон, Джон Влиссидес. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. – М.: Вильямс, 2019.
- [4] Кэти Сьерра, Берт Бейтс. Изучаем Java. – СПб.: Питер, 2021.
- [5] Oracle Corporation. Java Platform, Standard Edition Documentation. — Режим доступа: <https://docs.oracle.com> (дата обращения: 26.03.2026).
- [6] Oracle Corporation. JavaFX Documentation. – Режим доступа: <https://openjfx.io> (дата обращения: 26.03.2026).
- [7] Дональд Кнут. Искусство программирования. – М.: Вильямс, 2019.
- [8] Леонард Клейнрок. Теория массового обслуживания. – М.: Мир, 1979.
- [9] Oracle Corporation. Java Database Connectivity (JDBC) Documentation. - Режим доступа: <https://docs.oracle.com/javase/tutorial/jdbc/> (дата обращения: 31.05.2026).
- [10] SQLite Documentation. – Режим доступа: <https://www.sqlite.org.html> (дата обращения: 31.05.2026).
- [11] Fowler M. Patterns of Enterprise Application Architecture. - Boston: Addison-Wesley, 2003.

ПРИЛОЖЕНИЕ А

(Обязательное)

Листинг программного кода

Листинг А.1 – Содержимое файла orderwindow.java, отвечающее за управление заказами в системе

```
package ui;

import dao.Database;
import javafx.animation.FadeTransition;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.stage.Stage;
import javafx.util.Duration;
import java.sql.*;

public class OrderWindow {

    public static class ClientItem {
        int id; String fio; String address; double discount;
        public ClientItem(int id, String fio, String address, double
buyoutAmount) {
            this.id = id; this.fio = fio; this.address = address;
            this.discount = ClientWindow.calculateDiscount(buyoutAmount);
        }
        @Override public String toString() { return fio + " (Скидка: " +
discount + "%)"; }
    }

    public static class ProductItem {
        int id; String name; double price;
        public ProductItem(int id, String name, double price) { this.id = id;
this.name = name; this.price = price; }
        @Override public String toString() { return name + " (" + price + "
BYN)"; }
    }

    public static class OrderRow {
        private final int id;
        private final String clientInfo, productName, status, deliveryType,
deliveryAddress, comment;
        private final double totalPrice;

        public OrderRow(int id, String clientInfo, String productName, String
status, String deliveryType, String deliveryAddress, double totalPrice, String
comment) {
            this.id = id; this.clientInfo = clientInfo; this.productName =
productName; this.status = status;
            this.deliveryType = deliveryType; this.deliveryAddress =
deliveryAddress; this.totalPrice = totalPrice; this.comment = comment;
        }
        public int getId() { return id; }
        public String getClientInfo() { return clientInfo; }
    }
}
```



```

        public String getProductName() { return productName; }
        public String getStatus() { return status; }
        public String getDeliveryType() { return deliveryType; }
        public String getDeliveryAddress() { return deliveryAddress; }
        public String getTotalPriceDisplay() { return String.format("%.2f BYN",
totalPrice); }
        public String getComment() { return comment; }
    }

    private double calculatedFinalPrice = 0.0;

    public void show() {
        Stage stage = new Stage();
        stage.setTitle("Управление заказами");

        // Инициализация с подсказками
        ComboBox<ClientItem> clientBox = new ComboBox<>();
        clientBox.setPromptText("Выберите клиента из списка...");
        clientBox.setMaxWidth(Double.MAX_VALUE);

        ComboBox<ProductItem> productBox = new ComboBox<>();
        productBox.setPromptText("Выберите товар из каталога...");
        productBox.setMaxWidth(Double.MAX_VALUE);

        ComboBox<String> statusBox = new
        ComboBox<>(FXCollections.observableArrayList("Новый", "В обработке",
"Оплачен", "Доставлен", "Выполнен", "Отменен"));
        statusBox.setPromptText("Статус заказа");

        ComboBox<String> deliveryTypeBox = new
        ComboBox<>(FXCollections.observableArrayList("Самовывоз", "Доставка"));
        deliveryTypeBox.setPromptText("Способ получения");

        TextField addressF = new TextField(); addressF.setPromptText("Введите
адрес доставки");
        TextField commentF = new TextField();
        commentF.setPromptText("Комментарий к заказу");

        Label lblPriceCalculation = new Label("Итоговая стоимость: 0.00 BYN");
        lblPriceCalculation.setStyle("-fx-text-fill: #3498db; -fx-font-weight:
bold; -fx-font-size: 14px;");

        // Логика расчета
        Runnable recalculatePrice = () -> {
            ClientItem client = clientBox.getValue();
            ProductItem product = productBox.getValue();
            if (client != null && product != null) {
                calculatedFinalPrice = product.price * (1.0 - (client.discount
/ 100.0));
                lblPriceCalculation.setText(String.format("Итого: %.2f BYN (co
скидкой %.0f%%)", calculatedFinalPrice, client.discount));
            } else if (product != null) {
                calculatedFinalPrice = product.price;
                lblPriceCalculation.setText(String.format("Итого: %.2f BYN",
calculatedFinalPrice));
            }
        };

        clientBox.valueProperty().addListener((o, oldV, newV) ->
recalculatePrice.run());
        productBox.valueProperty().addListener((o, oldV, newV) ->
recalculatePrice.run());
    }

```

```

// Таблица
TableView<OrderRow> historyTable = new TableView<>();

historyTable.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY);
String[] cols = {"ID", "Клиент", "Товар", "Статус", "Тип", "Адрес",
"Сумма", "Комментарий"};
String[] props = {"id", "clientInfo", "productName", "status",
"deliveryType", "deliveryAddress", "totalPriceDisplay", "comment"};

for(int i=0; i<cols.length; i++) {
    TableColumn<OrderRow, String> col = new TableColumn<>(cols[i]);
    col.setCellValueFactory(new PropertyValueFactory<>(props[i]));
    historyTable.getColumns().add(col);
}

loadClients(clientBox);                                loadProducts(productBox);
loadOrdersHistory(historyTable);

Button btnSave = new Button("Оформить заказ");
btnSave.setMaxWidth(Double.MAX_VALUE);
btnSave.setOnAction(e -> {
    if (clientBox.getValue() == null || productBox.getValue() == null)
return;
    saveOrder(clientBox.getValue(),                productBox.getValue(),
statusBox.getValue() == null ? "Новый" : statusBox.getValue(),
deliveryTypeBox.getValue(), addressF.getText(), calculatedFinalPrice,
commentF.getText());
    loadOrdersHistory(historyTable);
});

// Панель формы
VBox formPane = new VBox(5,
    new Label("КЛИЕНТ:"), clientBox,
    new Label("ТОВАР:"), productBox,
    new Label("ДОСТАВКА:"), deliveryTypeBox,
    new Label("АДРЕС:"), addressF,
    new Label("СТАТУС:"), statusBox,
    new Label("КОММЕНТАРИЙ:"), commentF,
    lblPriceCalculation, btnSave);

VBox root = new VBox(15, formPane, new Separator(), historyTable);
root.setPadding(new Insets(20));
VBox.setVgrow(historyTable, Priority.ALWAYS);

Scene scene = new Scene(root, 1100, 700);
try                                                                 {
scene.getStylesheets().add(getClass().getResource("style.css").toExternalForm
()); } catch (Exception e) {}

stage.setScene(scene);
stage.show();

root.setOpacity(0);
FadeTransition ft = new FadeTransition(Duration.millis(400), root);
ft.setToValue(1); ft.play();
}

private void loadClients(ComboBox<ClientItem> box) {
    try (Connection conn = Database.connect(); ResultSet rs =
conn.createStatement().executeQuery("SELECT * FROM clients")) {
        while (rs.next()) box.getItems().add(new
ClientItem(rs.getInt("id"), rs.getString("fio"), rs.getString("address"),
rs.getDouble("buyout_amount")));
    }
}

```

```

        } catch (Exception e) { e.printStackTrace(); }
    }

    private void loadProducts(ComboBox<ProductItem> box) {
        try (Connection conn = Database.connect(); ResultSet rs =
conn.createStatement().executeQuery("SELECT * FROM products")) {
            while (rs.next()) box.getItems().add(new
ProductItem(rs.getInt("id"), rs.getString("name"), rs.getDouble("price")));
        } catch (Exception e) { e.printStackTrace(); }
    }

    private void loadOrdersHistory(Table<OrderRow> table) {
        ObservableList<OrderRow> data = FXCollections.observableArrayList();
        String sql = "SELECT o.id, c.fio, p.name, o.status, o.delivery_type,
o.delivery_address, o.total_price, o.comment " +
"FROM orders o JOIN clients c ON o.client_id = c.id JOIN
products p ON o.product_id = p.id";
        try (Connection conn = Database.connect(); ResultSet rs =
conn.createStatement().executeQuery(sql)) {
            while (rs.next()) {
                data.add(new OrderRow(rs.getInt("id"), rs.getString("fio"),
rs.getString("name"), rs.getString("status"),
rs.getString("delivery_type"),
rs.getString("delivery_address"), rs.getDouble("total_price"),
rs.getString("comment")));
            }
            table.setItems(data);
        } catch (Exception e) { e.printStackTrace(); }
    }

    private void saveOrder(ClientItem client, ProductItem product, String
status, String deliveryType, String address, double price, String comment) {
        try (Connection conn = Database.connect(); PreparedStatement ps =
conn.prepareStatement("INSERT INTO orders (client_id, product_id, status,
delivery_type, delivery_address, total_price, comment) VALUES
(?,?,?,?,?,?,?)")) {
            ps.setInt(1, client.id); ps.setInt(2, product.id); ps.setString(3,
status); ps.setString(4, deliveryType == null ? "Самовывоз" : deliveryType);
ps.setString(5, address); ps.setDouble(6, price); ps.setString(7, comment);
            ps.executeUpdate();
        } catch (Exception e) { e.printStackTrace(); }
    }
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Блок-схема общего алгоритма работы CRM-системы